EK AWAAZ

A safety-focused multilingual voice agent for emergency coordination

TEAM	Team Mentalist
MEMBERS	Atharv Bacchewar · Ayush Wable
SELECTED TRACK	Agentic AI
PROJECT TYPE	Voice-first emergency coordination demo

Ek Awaaz turns a multilingual emergency conversation into a structured, caller-confirmed incident and coordinates observable response actions through voice, SMS, and location sharing.

1. Problem Statement & Motivation

During an emergency, a caller may need to explain what happened, correct incomplete details, share a location, and contact several people while under stress. A voice conversation alone does not reliably create a verified incident or a coordinated action plan. This gap becomes more important when the caller speaks in Hindi, Hinglish, Marathi, or English—or cannot safely speak at all.

Ek Awaaz addresses this problem with a human-confirmed agentic workflow: it listens, structures the situation, exposes uncertainty, asks for confirmation, and only then coordinates configured response actions.

2. Target Users & Intended Use Cases

- A caller reporting an accident, medical emergency, hazard, or unsafe situation.
- Trusted family contacts and configured responder stand-ins receiving updates.
- An operator monitoring the live console, confidence, corrections, and action status.
- Judges and evaluators running deterministic spoken and silent-mode demos.

3. Proposed Solution & Key Features

- Multilingual voice-first intake with rules-first extraction and optional Groq structured extraction.
- Correction-aware incident facts for location, injuries, hazards, landmarks, and urgency.
- Explicit caller confirmation before any dispatch action; material corrections reset confirmation.
- LangGraph-based planning with parallel, observable actions and bounded retries.
- Bolna voice calls, mock-by-default SMS, optional Twilio SMS, and browser GPS sharing.
- Silent DTMF mode, WebSocket live console, searchable reports, and downloadable PDF summaries.

4. Technical Approach

Ek Awaaz uses one shared orchestration path for simulator, manual transcript, and Bolna webhook input. Incident state and event history are persisted so the operator can see what the agent understood, what changed, and which actions were attempted.

> Caller / simulator -> FastAPI transcript intake -> rules + optional Groq extraction -> SQLite incident state -> confirmation gate -> LangGraph dispatcher -> Bolna stand-in calls / mock-or-Twilio SMS -> React console + report

Technology stack

Layer	Tools / implementation
Interface	React, Vite, WebSockets, responsive operations console
Backend	Python 3.11.9, FastAPI, SQLite, Pydantic, ReportLab
Agentic workflow	LangGraph state machine, confirmation gate, parallel dispatch, retries, kill switch
AI and integrations	Optional Groq extraction, Bolna voice API/webhooks, optional Twilio SMS
Reproducibility	Pinned runtimes, npm lockfile, hash-pinned Python lockfile, Docker, make test

5. How the Agent Acts

- Perceive: receive natural speech, transcript turns, DTMF input, or browser location.
- Structure: extract incident facts and confidence; retain correction history.
- Reason: check completeness and plan tailored actions using stateful LangGraph orchestration.
- Authorize: ask the caller to confirm the interpreted incident before side effects.
- Execute: run allowlisted voice/SMS actions with status events, rate limits, and retry bounds.
- Account: persist an event timeline and generate an operator-readable report.

6. Feasibility & Implementation Plan

Prototype already demonstrated

- Deterministic accident demo with a location correction.
- Silent medical-emergency DTMF demo.
- Live WebSocket console and PDF report generation.
- Mock-safe actions, confirmation gating, and emergency-number blocking.

Next implementation steps

- Harden multilingual scenarios and low-confidence failure states.
- Validate provider webhooks and idempotency with owned stand-in numbers.
- Add privacy retention, access control, and structured audit logs.
- For production scale, migrate SQLite events to a durable multi-instance store and queue.

7. Innovation & Differentiation

The project demonstrates controlled agency rather than an opaque chatbot. Its distinctive value is the complete loop from conversation to verified state to observable action:

- Confirmation is a first-class authorization boundary, not a final UI prompt.
- A material correction invalidates earlier confirmation, reducing stale-action risk.
- Side effects are kept outside graph nodes so safety controls remain centralized.
- Spoken and silent interaction modes support different emergency constraints.
- The action timeline makes every interpretation, correction, retry, and cancellation inspectable.
- Provider adapters keep the orchestration layer portable across simulator, manual, and live inputs.

8. Expected Impact & Scalability

Expected impact

- Reduce the cognitive burden on a caller under stress.
- Make multilingual emergency intake more structured and reviewable.
- Help one caller coordinate multiple trusted contacts.
- Create an auditable record for follow-up and evaluation.

Scalability path

- Keep provider integrations behind adapters and preserve one shared agent workflow.
- Replace local SQLite with a durable store and queue for multi-instance operation.
- Add idempotent callbacks, observability, privacy controls, and supervised pilot testing.
- Extend integrations only after safety and regional requirements are validated.

9. Prototype & Submission Links

GitHub repository	github.com/Atharva14518/ek-awaaz
Demo video	Open the recorded demo on Google Drive

Safety-by-design

The prototype is demo-safe by default: external actions are simulated, real actions require independent safety switches and explicit confirmation, recipients are allowlisted, and Indian emergency numbers are blocked. Ek Awaaz is positioned as a supervised coordination layer—not a replacement for emergency services.

> **Proposal thesis: Agentic AI can do more than transcribe an emergency call—it can build situational understanding, ask for confirmation, plan coordinated actions, expose its state to operators, and preserve an accountable record.**